Enhanced LLM-Aided Testbench Generation

Coverage-guided test pattern refinement with Icarus Verilog + Covered

Course: LLM4ChipDesign | Instructors: Ramesh Karri, Weihua Xiao

Overview

The original workflow automatically generates a stimulus-only testbench, creates a Python golden reference model from the natural-language specification, labels the test patterns with expected outputs, and produces a final self-checking Verilog testbench. In this assignment, you will extend that workflow with a **coverage-guided feedback loop** so that test-pattern quality is evaluated and improved using Icarus Verilog and Covered rather than relying only on the LLM.

Updated pipeline

1. RTL + NL spec	**2. LLM initial patterns**	**3. Stimulus-only TB**
6. LLM targeted patterns	**5. Parse uncovered targets**	**4. iverilog/vvp + Covered**
7. Coverage-optimized patterns	**8. Python golden model**	**9. Final self-checking TB**

Key idea: optimize the stimulus first. Coverage answers whether the test patterns exercise the RTL; the Python golden model separately answers what the correct outputs should be. High coverage does not by itself prove functional correctness.

Why a more complex ALU is included

The notebook tutorial examples (2-to-1 mux, 4-bit adder, and binary_to_bcd) are useful for learning the pipeline, but they are small enough that random or LLM-generated patterns may reach high coverage quickly. The main graded experiment therefore uses a provided **8-bit multi-function ALU** with arithmetic, logic, shift, comparison, flag, and default behaviors. It is large enough to create meaningful coverage gaps but still small enough to understand and model in Python.

Learning goals

- Measure line, toggle, and combinational-logic coverage using **iverilog/vvp + Covered**.
- Parse coverage reports into machine-readable uncovered targets.
- Build a closed loop: **generate - simulate - measure coverage - refine - repeat**.
- Use a Python golden reference model to label the final coverage-optimized patterns.
- Compare baseline LLM-only patterns with coverage-guided patterns for bug detection.

Part I - Reproduce the Icarus Verilog + Covered coverage demo

Use the provided **coverage_exp_classroom** materials as the reference flow before integrating coverage into the LLM pipeline.

1. Run **student_coverage_demo.ipynb** from top to bottom, or run **make report** inside the coverage_exp directory.
2. Compile and simulate **src/simple_alu.v** with **tb/simple_alu_tb.v**.
3. Generate the VCD, score it with Covered, and produce **coverage_summary.rpt** and **coverage_detail.rpt**.
4. Confirm that the original testbench exercises ADD and AND but deliberately misses OR and XOR.
5. Add targeted OR and XOR stimuli, rerun the flow, and compare the reports.

Reference commands

```
iverilog -g2001 -Wall -s simple_alu_tb -o build/simple_alu.vvp src/simple_alu.v
tb/simple_alu_tb.v
vvp build/simple_alu.vvp
covered score -t simple_alu -i simple_alu_tb.dut -v src/simple_alu.v -vcd
build/simple_alu.vcd -o build/simple_alu.cdd
covered report -m ltc -d s -o build/coverage_summary.rpt build/simple_alu.cdd
covered report -m ltc -d v -i -o build/coverage_detail.rpt build/simple_alu.cdd
```

Baseline values in the classroom example

Metric	Baseline result
Line coverage	3/5 = 60%
Toggle 0->1	9/14 = 64%
Toggle 1->0	3/14 = 21%
Combinational logic	6/17 = 35%

Part II - Warm-up with the original LLM-only pipeline

Choose **one** notebook tutorial example (2-to-1 mux, 4-bit adder, or binary_to_bcd) and run the original pipeline once. This is a warm-up; the 8-bit ALU in Part III is the main coverage-guided task.

- Generate the initial stimulus-only testbench / patterns with the LLM.
- Generate the Python golden reference model from the natural-language specification.
- Generate expected outputs and produce the final self-checking testbench.
- Compile and simulate the final testbench with iverilog/vvp and show the result.

Part III - Main challenge: provided 8-bit multi-function ALU

Use the provided file **coverage_guided_alu.v** and the specification below. Do not simplify the RTL or remove operations. Your objective is to automatically generate and refine test patterns for this DUT.

Interface

Signal	Dir.	Width	Meaning
a	input	8	operand A
b	input	8	operand B / shift amount
op	input	4	operation selector
y	output	8	result
zero	output	1	1 when y == 0
negative	output	1	y[7]
carry	output	1	ADD carry-out; SUB no-borrow flag
overflow	output	1	signed ADD/SUB overflow

Operation table

op	Operation	Behavior
0x0	ADD	y = a + b
0x1	SUB	y = a - b
0x2	AND	y = a & b
0x3	OR	y = a | b
0x4	XOR	y = a ^ b
0x5	NOT	y = ~a; b ignored
0x6	SLL	y = a << b[2:0]
0x7	SRL	y = a >> b[2:0]
0x8	SRA	signed(a) arithmetic-right-shift by b[2:0]
0x9	SLT	signed compare; y=1 if signed(a)<signed(b), else 0
0xA-0xF	DEFAULT	y=0, carry=0, overflow=0

Flag semantics and useful corner cases

- **zero** is 1 whenever y is 0; **negative** is y[7].
- **carry** is the ADD carry-out. For SUB, carry=1 means no unsigned borrow (a >= b). For other operations, carry=0.
- **overflow** is signed two's-complement overflow for ADD/SUB and 0 otherwise.

- Important corner cases include ADD carry, ADD/SUB signed overflow, SUB borrow/no-borrow, zero/negative outputs, shift amounts 0/1/7, arithmetic shift of negative values, signed SLT around 0x80/0x7F, and unsupported opcodes.

Do not hand-write a complete directed test suite before measuring coverage. The initial pattern set should come from the LLM without access to the coverage report. The point of the assignment is to let the coverage loop discover what the initial patterns missed.

Part IV - Establish the LLM-only ALU baseline

1. Provide **coverage_guided_alu.v** and the natural-language ALU specification to your test-pattern generator.
2. Ask the LLM to generate an initial set of approximately **12-20 stimulus patterns**. Do not provide any coverage feedback yet.
3. Generate a stimulus-only testbench that writes a VCD and can be simulated with iverilog/vvp.
4. Run Covered and save **coverage_summary_iter0.rpt** and **coverage_detail_iter0.rpt**.
5. Save the initial patterns as **alu_test_patterns_initial.json** and record the baseline coverage metrics.

Part V - Implement coverage-guided pattern refinement

Extend the pipeline so that the detailed Covered report becomes structured feedback for the LLM.

1. Parse the summary report to extract line coverage, toggle 0->1, toggle 1->0, and combinational-logic coverage.
2. Parse the detailed report to identify useful targets such as missed RTL lines, incomplete input/output toggles, and missed logic combinations.
3. Build a compact prompt containing the ALU specification, current pattern set, coverage metrics, and uncovered targets.
4. Ask the LLM to generate **additional targeted patterns only**. Do not delete existing patterns.
5. Regenerate the stimulus-only testbench, rerun simulation + Covered, and record the new reports.
6. Repeat until the stopping condition is met.

Recommended stopping conditions

- Line coverage is near-complete and other coverage metrics have improved substantially; or
- Coverage improvement becomes negligible (for example, less than about 1-2 percentage points); or
- A maximum of **3-5 refinement iterations** has been reached.

Coverage-feedback prompt requirement

For every refinement iteration, require the LLM to return both (1) the new patterns and (2) a short mapping from each new pattern to the uncovered behavior it is intended to exercise. This makes the refinement interpretable rather than simply increasing the number of random tests.

Example feedback items: **op=0x8 SRA never executed**; **ADD signed-overflow path not exercised**; **result[7] never toggles 0->1**; **default opcode behavior is uncovered**.

Part VI - Generate golden outputs after coverage optimization

Only after the coverage loop has finished should you generate expected outputs.

- Generate **golden_model.py** from the natural-language ALU specification.
- Run the golden model on every pattern in **alu_test_patterns_coverage_optimized.json**.
- Create **alu_test_patterns_with_golden.json** containing stimulus and expected y/flag outputs.
- Generate **alu_testbench_final.v** with expected values, comparisons, and pass/fail reporting.
- Compile and simulate the final testbench using iverilog/vvp and show the final result.

Part VII - Compare bug detection before and after coverage refinement

Introduce **one small intentional bug** into a copy of the ALU RTL. Suitable examples include making SRA behave like SRL, using unsigned instead of signed comparison for SLT, or introducing an error in ADD/SUB flag behavior.

- Run the buggy DUT with the **baseline LLM-only pattern set** and record whether the bug is detected.
- Run the same buggy DUT with the **coverage-optimized pattern set** and record whether the bug is detected.
- Explain whether coverage-guided patterns improved bug-detection effectiveness and why.
- Restore the correct DUT and show the final passing self-checking simulation.

Conceptual note

Coverage is a stimulus-quality signal, not a correctness oracle. A test suite can reach high structural coverage and still miss functional bugs. Covered tells you what RTL structures were exercised; the Python golden model tells you whether the observed outputs are correct.

Required generated artifacts

Use clear names. A recommended organization is:

```
alu_test_patterns_initial.json
alu_testbench_initial.v
coverage_summary_iter0.rpt
coverage_detail_iter0.rpt
alu_test_patterns_iter1.json
coverage_summary_iter1.rpt / coverage_detail_iter1.rpt
...
alu_test_patterns_coverage_optimized.json
coverage_history.json
golden_model.py
alu_test_patterns_with_golden.json
alu_testbench_final.v
```

Coverage history

Your **coverage_history.json** should record at least the iteration number, number of patterns, and the coverage metrics available from Covered. Include a table or plot in your report showing how coverage changes across iterations.

What to submit

Submission method:

- **Undergraduate (UG):** submit through **Brightspace**. No GitHub or email submission is required.
- **Master's (MS):** submit through your **GitHub homework repository** (no email submission). Create a folder such as **HW_TestbenchGen_Coverage**/.
- **Executed notebook:** your updated end-to-end notebook with visible outputs.
- **Provided ALU files:** include the supplied **coverage_guided_alu.v** and the natural-language specification used by your pipeline.
- **Coverage integration code:** scripts/functions used to run Icarus Verilog, Covered, parse reports, and generate coverage-feedback prompts.
- **Generated artifacts:** baseline, intermediate, and final ALU pattern/coverage/testbench files.
- **Report PDF: CoverageGuided_TestbenchGen_Report.pdf**.
- **MS students only - Repo access:** invite **weihuax6@gmail.com** as a collaborator.

Report requirements

- **Pipeline overview**: explain why coverage refinement occurs before golden-output generation.
- **Coverage demo**: show the simple_alu baseline and the result after adding OR/XOR tests.
- **Warm-up example**: briefly summarize one original notebook example and its final self-checking result.
- **8-bit ALU baseline**: report the initial patterns, baseline coverage, and the main uncovered targets.
- **Coverage-guided trajectory**: show iteration-by-iteration coverage and explain the targeted patterns added at each iteration.
- **Golden model**: explain how the final pattern set is labeled with expected y/flag outputs.
- **Bug experiment**: compare bug detection using baseline versus coverage-optimized patterns.
- **Limitations**: discuss at least one case where higher coverage does not necessarily imply better functional verification.

Grading rubric

Criterion	Points
Coverage tutorial + correct Icarus/Covered flow	10
Original-pipeline warm-up example	10
8-bit ALU baseline + report parsing	15
Coverage-guided ALU refinement loop	30
Golden model + final self-checking ALU testbench	15
Bug-detection comparison	10
Report clarity + reproducibility	10

Academic integrity and responsible use

Using an LLM is required for this assignment, but you must submit your own work. Do not share completed code or final prompts with other teams. Always verify LLM-generated artifacts with simulation and coverage tools. Do not include API keys or secrets in your submission.